

流程：七個檢查對六個原則，不過就做 patch，由專案 owner 決定收不收

這份文件回答：人或 AI agent 進到一個沒看過的 workspace，怎麼檢查它有沒有 CI/CD；不過時自己先補什麼 ↓

七個檢查，照這個順序

每個檢查都走一遍

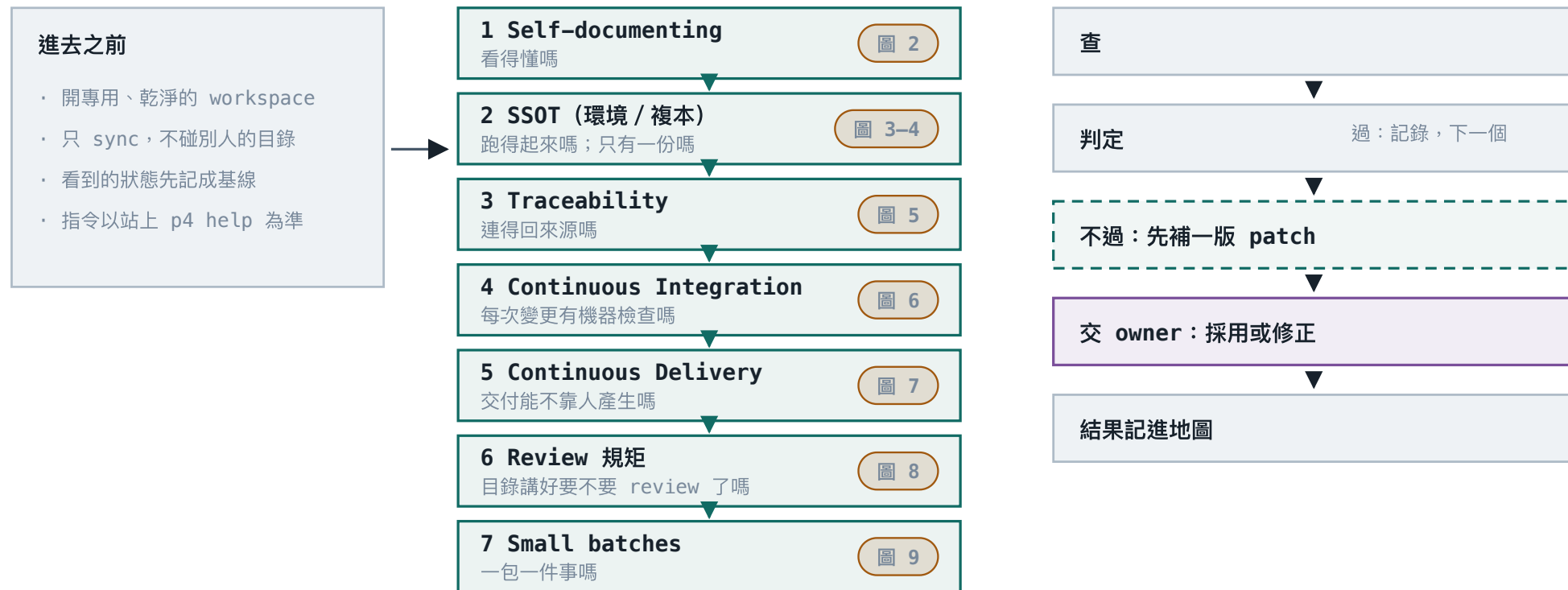


圖 6 的 check 直接用圖 3 的 script；圖 7 的出包用圖 5 的 manifest

七個檢查對應六個原則的檢驗 (review 規矩歸 Self-documenting)；不過的時候，多數缺口靠通用常識就能先補一版。

owner 拿到的是可以直接採用的 patch，只有少數幾件事需要他回答。

Self-documenting：光看 depot 說不出每個目錄做什麼，就先補一份目錄說明

Self-documenting

檢驗：第一次來的人或 AI agent 只讀 repo，就說得出每個目錄的用途與相依

怎麼查

- 列頂層目錄，找 README、說明檔
- 找 filelist 與 run script
- 對照：哪些檔案沒被任何 filelist 引用
- 看最近的 CL：哪些目錄還有人在動

判定

- ✓ 每個頂層目錄說得出用途
- ✓ 誰引用誰畫得出來
- ✓ 怎麼跑有一份寫下來的

不過的徵兆

- x rtl_old/ 與 rtl_new2/ 並存
- x README 寫的是上一個專案
- x 沒有任何檔案說怎麼跑

不過時，先補這些（自己做）

PROJECT_MAP.md 初稿
每個目錄一行用途，標依據：擷取 / 推測 / 猜的

相依圖
從 filelist 擷取誰引用誰

疑似未使用清單
沒被引用也沒人動的目錄，只標不刪

只需要問 owner

- ? 標「猜的」那幾行對不對
- ? 疑似未使用的目錄能不能標為棄用

看懂的依據要分開標：擷取的、推測的、猜的，owner 只需要看猜的那幾行。

沒被引用的目錄先標、不刪；刪不刪是 owner 的事。

SSOT (環境)：乾淨的 workspace 跑不起來，缺的先補成 script 進 depot

Single Source of Truth (SSOT)

檢驗：換一台乾淨的機器，只靠版控的內容做出同一個結果

怎麼查

- 專用 workspace 只 sync，不帶本機設定
- 照 repo 裡寫的方式編 top、跑一個最小的 sim
- 每個失敗分類：缺檔 / 缺工具 / 缺環境 / 缺 script
- 跟 owner 手上的結果比對

判定

- ✓ 編得過、跑得完
- ✓ 結果和 owner 手上的一致

不過的徵兆

- ✗ filelist 引用不存在的檔
- ✗ script 在 /home 或 /proj 底下
- ✗ 工具版本只在 .cshrc 裡
- ✗ 同一份 RTL，兩台機器結果不同

不過時，先補這些（自己做）

setup.sh

工具版本、環境變數寫死在裡面，進 depot

run_sanity.sh

從乾淨 workspace 編 top+跑一個 sim

缺檔清單

哪個 filelist 第幾行引用、誰最後動過

.p4ignore 提案

進了 depot 的產生檔列出來

只需要問 owner

? 缺的檔在誰的 workspace

? 產生檔要不要移出版控

跑不起來的每一個原因，都是一件散在 depot 外面的東西；補法是把它寫成 script 進 depot。

只有「缺的檔在誰那裡」這件事非問 owner 不可。

SSOT (複本)：產物、複製的 flow、解壓的 IP 各只留一份來源，其餘改成產生

Single Source of Truth (SSOT)

檢驗：換一台乾淨的機器，只靠版控的內容做出同一個結果；產物、flow、IP 也算在內

怎麼查

- 找進了 depot 的產物：netlist、lib、sim 結果
- 比對產物和來源：產物比來源新，就是直接改過
- 找 flow 的複本：各專案的 scripts/ 差在哪
- 找第三方 IP：怎麼進來的，有沒有 drop 的紀錄

判定

- ✓ 產物都能從來源重新產生
- ✓ flow 只有一份，差異是參數
- ✓ 每次 IP drop 都查得到版本與差異

不過的徵兆

- x ECO 改在 netlist 上
- x 三個專案三份 flow
- x ip/ 是解壓覆蓋的，沒有紀錄

不過時，先補這些（自己做）

產物清單+重新產生的 script

列出哪些是產物、由哪個來源產生

flow 收成一份

三份的差異做成參數，各專案指向同一份

IP drop 流程

每次 drop 一個 CL：tarball 指紋、版本、差異

本地 patch 分開放

對 IP 的修改獨立成 patch，drop 之後重套

只需要問 owner

- ? 產物能不能從 depot 移出
- ? flow 的正本由誰維護
- ? IP 的本地修改要不要回報 vendor

產物、flow、IP 最常變成第二份真相；補法都一樣：留一份來源，其他的由它產生。

移出產物、收攏 flow 會動到別人的工作方式，要 owner 決定。

Traceability：結果說不出哪個 CL 跑的，就附一份 manifest 記來源

Traceability

檢驗：隨便拿一份結果，說得出它的版本、工具、環境與步驟

怎麼查

- 拿最近的 label、release 包、regression 報告
- 問四件事：CL、工具版本、環境、怎麼跑
- 用內容指紋比對 release 包和 label 的檔案
- 試著退回上一個能跑的狀態

判定

- ✓ 四件事都從紀錄答得出，用不著問人
- ✓ release 包和 label 對得上
- 不過的徵兆
 - x label 只有檔案清單
 - x 報告是一個會變的路徑
 - x release 包手動打，README 靠回想
 - x 退回 label，環境和產物回不去

不過時，先補這些（自己做）

make_manifest.sh
寫出 CL、label、工具版本、環境、指令、結果摘要

manifest 跟著結果走
放進 run 目錄與 release 包，一起進 depot

舊交付物反查
指紋對得上就補 manifest，對不上標「無法追溯」

known-good 點
check 通過就打 label+manifest，要回去有地方回

只需要問 owner

? 交付物的正式清單：給誰、含什麼

manifest 是結果和來源之間缺的那條線；讓產生結果的 script 順手寫出來，就不用靠人記。

過去的交付物追不回來的，標成基線，從現在開始追。

CI：submit 後沒有機器檢查，就先掛一個 sanity check 跟著 submit 跑

Continuous Integration (CI)

檢驗：改壞的那一包進來時就被標出來，用不著等到整合

怎麼查

- 找 submit 觸發的檢查：p4 triggers、CI job
- 找定期 regression：結果放哪、誰看得到
- 翻最近一次壞掉：多久後、被誰發現

判定

- ✓ 每個 submit 都有機器跑過最小檢查
- ✓ 結果公開看得到，附 CL 號

不過的徵兆

- ✗ 只有人手動跑
- ✗ 有 nightly，結果只在跑的人信箱
- ✗ 壞掉由下游先發現
- ✗ 狀態表靠人填 Excel

不過時，先補這些（自己做）

最小 check

拿圖 3 的 run_sanity.sh：編得過＋一個 sim

排上去跑

先定時跑，再進到每個 CL 都跑

結果可見

寫到 repo 看得到的地方，附 CL 號與 manifest

狀態表由結果產生

從 manifest 與 log 產生，不用人填

只需要問 owner

？觸發要裝在哪、誰有權限

？算力與 license 的配額

？壞了通知誰

最小的 check 就是圖 3 那個 script；先讓它定時跑、結果看得到，再談每個 CL 都跑。

裝觸發、配算力、通知誰，要 owner 點頭。

CD：交付靠 email 貼路徑，先幫 owner 定出交付物，出包做成 script 跟著 check 跑

Continuous Delivery (CD)

檢驗：任何時候不用問人，就拿得到最新一份附 manifest 的交付包，下游拿了就能跑

怎麼查

- 找交付物：label、release 目錄、下游引用的路徑
- 問下游：從哪裡拿、怎麼知道更新了
- 看 release 怎麼出：script 還是人手打包

判定

- ✓ 交付物有定義：交什麼、給誰、什麼形式
- ✓ 出包是 script，main 過 check 就跑
- ✓ 下游從固定位置拿，不用等通知

不過的徵兆

- ✗ 交付靠 email 貼路徑
- ✗ release 包手動打、README 靠回想
- ✗ 下游不知道哪一版能用

不過時，先補這些（自己做）

交付物草稿

從 CL 歷史、label、下游引用推出：交什麼、給誰、什麼形式

make_release.sh

打包+manifest+label+放到取用處

取用處與通知

release/<目錄>/ 一個固定位置；出包時通知下游

只需要問 owner

- ? 交付物的定義對不對（owner 多半沒想過，給草稿）
- ? 取用處放哪、誰能寫

owner 多半沒有「交付物」的概念：agent 先推出草稿給他確認，再把出包做成 script，main 過 check 就自動跑。

這一步做完，中圈的交接才不用等人；下游拿了就能跑。

Review 規矩：目錄沒講好要不要 review，先補一份規矩初稿給 owner 定

Review 規矩（每個目錄講好要不要 review）

檢驗：隨便挑一個目錄，說得出它要不要 review、誰看；說要的目錄，隨便挑一包說得出誰看過

怎麼查

- 目錄有沒有寫要不要 review：PROJECT_MAP
- 找 reviewer 的紀錄：Swarm、reviewed-by
- 找兩人改同檔的 resolve：誰看過差異

判定

- ✓ 每個目錄講好了：要 / 不要、誰看、什麼時候
- ✓ 說要的目錄，每一包併入前有人看過並留紀錄

不過的徵兆

- ✗ 沒有任何目錄講過要不要 review
- ✗ submit 就算完成
- ✗ resolve 直接 accept 整份

不過時，先補這些（自己做）

review 規矩初稿

每目錄：要 / 不要、誰看、何時；寫進 PROJECT_MAP

最輕的流程

說要的：shelve → 指定 reviewer → 看過才 submit

resolve 清單

最近整份 accept 的 resolve，列給當事人看

只需要問 owner

? 每個目錄要不要 review、誰看

? 要不要擋 submit (trigger)

要不要 review 由各目錄自己定、可以改，但要講好、寫下來；沒講好等於沒人看。

agent 交的 shelved CL 一律 owner 收了才進 depot，這條不由目錄定。

Small batches：過去的大 CL 改不了，先給 CL 說明模板和拆小 CL 的示範

Small batches

檢驗：隨便挑一包 submit，說得出它改了哪一件事

怎麼查

- p4 changes 看最近的 submit：幾個檔案、說明
- 挑幾包用 p4 describe 看：一包裡幾件事
- 看兩包之間隔多久

判定

- ✓ 一包一件事
- ✓ 說明看得懂改了什麼、為什麼

不過的徵兆

- ✗ 說明是 update、fix、sync
- ✗ 一包混 RTL、tb、script
- ✗ 一包是某人一整段期間的工作

不過時，先補這些（自己做）

CL 說明模板

改了什麼 / 為什麼 / 怎麼驗

pre-submit 清單

跑過 sanity check、filelist 更新、一包一件事

拆包示範

拿一包進行中的工作，用 shelved CL 拆成幾包

只需要問 owner

？團隊願不願意用模板

？進行中的大包怎麼拆，當事人決定

submit 的歷史改不了，能補的是下一包的寫法：模板、清單、一個拆開的示範。

大包怎麼拆，只有當事人知道哪些改動是一起的。

分工：agent 能補的直接送 patch，只有 owner 才知道的事才開口問



patch 怎麼交



從頭到尾守住的



多數缺口靠通用常識就補得出第一版；owner 的工作從回答問題變成採用或修正 patch。

三類事只有 owner 知道；問的時候附上你已經查到的部分。